

2025 -2026

Conception de Bases de Données & Bases de Données relationnelles

MSI M1
GROUPE B

Présenté par

Lina SAFI 12117731

Laura DAGUET 12506085

Ashani GOBALASAMY 12200679

Eunice KOUASSI 12519022



Table des matières

INTRODUCTION.....	2
PARTIE 1 – CONCEPTION BASE DE DONNEES	3
ANALYSE DE L’EXISTANT : MEA	3
PROBLEMES IDENTIFIES DANS LE MODELE ACTUEL	4
NOUVEAU MEA OPTIMISE.....	6
Explication du nouveau MEA	6
MODELE LOGIQUE RELATIONNEL (MLD).....	7
PARTIE 2 – BASE DE DONNEES.....	8
Q4 - JUSTIFICATION DES REQUETES	8

Figure 1- MEA de l'existant via l'éditeur Canva 3

Figure 2 - MEA optimisé via l'éditeur drawio.io **Erreur ! Signet non défini.**

Introduction

Dans le cadre du cours de Conception de bases de données et Base de données relationnelles, ce dossier rend compte de la modélisation et de l'implémentation d'une base dédiée à l'association AFIHM, portant sur la refonte et la mutualisation de données actuellement dispersées (gestion des membres et de leurs cotisations, facturation, et inscriptions/conférences) pour offrir un schéma cohérent, durable et exploitable par les trésoriers comme par les organisateurs des conférences IHM (calculs des montants, suivi des inscriptions, listes des participants, contraintes d'alimentation, etc.).

Notre démarche s'inscrit dans la méthode enseignée : à partir de l'analyse de l'existant et des règles de gestion, établir un modèle entité-association correct (normalisation des entités et associations, dépendances fonctionnelles/CIF, contraintes de spécialisation et d'intégrité), puis le traduire en modèle logique relationnel avant l'implémentation physique sous MySQL, me permettant de respecter la progression MEA → MLD → MPD, au cœur de la méthode MERISE, garantissant la cohérence sémantique et l'intégrité référentielle tout en préparant efficacement les requêtes analytiques ciblées par les demandeurs.

Le présent document se structure pour restituer clairement les raisons de nos choix : synthèse s'attachant à critiquer les problèmes du schéma actuel, proposition d'un modèle conceptuel unifié assortie des justifications, puis passage au schéma relationnel et mise en œuvre.

Partie 1 – Conception base de données

Analyse de l'existant : MEA

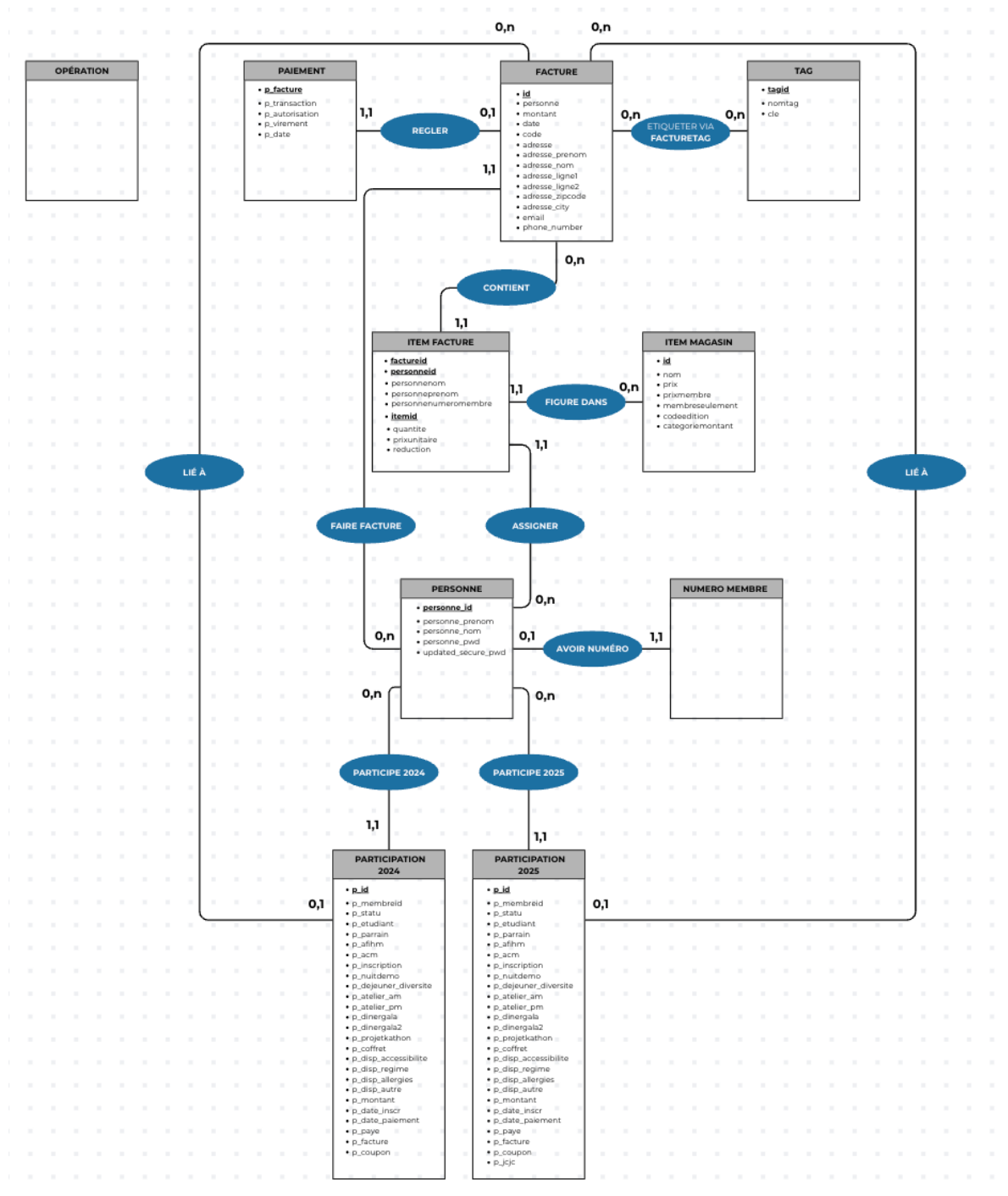


Figure 1- MEA de l'existant via l'éditeur Canva

Problèmes identifiés dans le modèle actuel

1. Dupliquer une BD par année (participant2024, participant2025, ...)

- Problème : Le fait de dupliquer la base de données chaque année et d'ajouter de nouvelles colonnes viole le principe de stabilité. L'intégrité référentielle n'est pas garantie et le modèle n'est pas non plus générique.
- Bonne pratique : La stabilité (modèle indépendant du temps), un seul MCD puis un seul MLD pour tout le domaine ; éviter les schémas « qui changent avec les données ».

2. Booléens de participation (p_nuitdemo, p_atelier_am/pm, p_jcjc, ...)

- Problème : Certains champs décrivent des participations spécifiques à un événement donnée dans des colonnes. Nous retrouvons des incohérences.
- Bonne pratique : Nécessite une nouvelle entité. Une association $n \leftrightarrow n$ "PARTICIPE_A" entre INSCRIPTION et ÉVÉNEMENT est la forme correcte.

3. NUMÉRO DE MEMBRE attribué à toute personne inscrite

- Problème : Le modèle actuel attribue un numéro de membre à toutes les personnes inscrites à une conférence, même si aucune cotisation n'a été payée, ce qui dénature la fonction sémantique de l'attribut « numéro_membre » dont le rôle est d'identifier un adhérent cotisant. Cette pratique force l'entité générale PERSONNE à contenir un attribut pertinent uniquement pour le sous-ensemble des MEMBRES, ce qui rend l'information inadapté pour les utilisateurs simples.
- Bonne pratique : Spécialisation sur-type → sous-type PERSONNE → MEMBRE, avec une contrainte (I/T/X) entre sous-types.

4. FACTURETAG (une facture = un "tag")

- Problème : Une facture ne peut contenir qu'un seul type d'article (adhésion ou inscription). Ce qui ne permet pas de facturer plusieurs types de produits (adhésion + inscription + places gala).
- Bonne pratique : Besoin d'une LIGNE_de_FACTURE (relation d'association 1,n depuis FACTURE vers PRODUIT).

5. PAIEMENTS : PK = p_facture

- Problème : Dans le modèle, il y a au plus un paiement par facture. Ce qui signifie qu'il n'y a pas de paiements partiels ou remboursements.
- Bonne pratique : Cardinalités et CIF : FACTURE (1) —règle→ (0,n) PAIEMENT.

6. Redondances et violations 3FN

- Exemples :
 - FACTURE : Les adresses et ses décompositions sont dans une même table, ce qui crée une duplication d'information.
 -ITEMFACTURE : On retrouve une copie des nom/prénom/numéro_membre alors que FACTURE→PERSONNE suffit (CIF), (dépendances transitives).
- Bonne pratique : 1FN/2FN/3FN : Des attributs élémentaires, pour qu'il n'y ai pas de dépendances vers d'autres attributs que la clé.

7. **ITEMMAGASIN.codeedition est un ENUM listant les années ('', 'ihm2015', ...)**

- Problème : Anti-pattern temporel (ajout de valeurs = changement de schéma / '' utilisé comme valeur) : Ces éléments crée une instabilité.
- Bonne pratique : Une séparation des référentiels (CONFERENCE comme entité).

8. **ATELIER non modélisé (p_atelier_am/pm entiers)**

- Problème : Il n'y a pas de clé étrangère ni de cardinalité, ce qui fait que l'intégrité n'est pas garantie.
- Bonne pratique : Il faut créer l'entité EVENEMENT (+ sous-type ATELIER par contrainte) et l'association n↔n avec INSCRIPTION.

9. **COUPON non modélisé**

- Problème : Le champ p_coupon existe mais il n'y a aucune table de référence, ni aucunes règles d'usage (code, type, validité, etc.). Ce qui signifie qu'il est impossible d'obtenir la liste fiable des utilisateurs de manière « propre ».
- Bonne pratique : Il faut créer une entité référentielle + clé étrangère

10. **Adresses et montants**

- Problème : Nous retrouvons plusieurs informations répétées
 Des adresses : Adresse (bloc) et adresse_ligne1/2/zipcode/city dans FACTURE
 Des montants en INT (11) sans décimales, ce qui crée une fragilité.
- Bonne pratique : Règles 1FN/3FN. Il faut utiliser des types adaptés (tel que DECIMAL (10) + devise) ; mais la photographie des données de facturation se prend dans FACTURE (attributs « stables à l'instant d'émission »).

11. **Table OPÉRATION**

- Les spécifications sont absente (colonnes non listées). Il est impossible d'en garantir l'usage et les liens.

Nouveau MEA optimisé

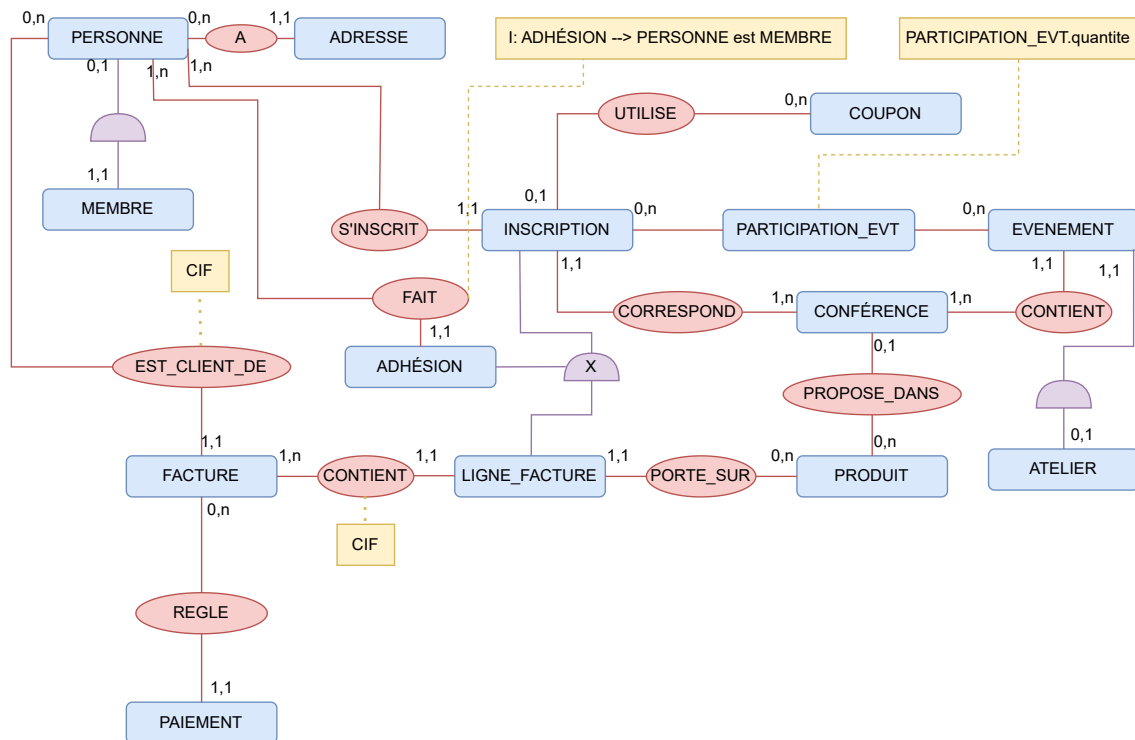


Figure 2 - MEA optimisé via l'éditeur drawio.io

Explication du nouveau MEA

Le modèle proposé repose sur un sur-type PERSONNE et un sous-type MEMBRE. Le numéro de membre et les cotisations sont gérés via l'entité ADHESION (début/fin stables), ce qui clarifie la sémantique et évite d'attribuer un numéro de membre à toute personne inscrite.

Côté conférence, une entité CONFERENCE regroupe les EVENEMENT (CIF), dont ATELIER est un sous-type. L'inscription d'une personne à une conférence est portée par INSCRIPTION ; la participation à des événements (gala, ateliers AM/PM, nuit des démos...) est modélisée par l'association PARTICIPATION_EVT ($n \leftrightarrow n$), ce qui remplace les anciens booléens.

La facturation est normalisée : une FACTURE contient plusieurs LIGNE_FACTURE rattachées à des PRODUIT ; une facture peut être réglée par plusieurs PAIEMENT. Une LIGNE_FACTURE référence soit une INSCRIPTION soit une ADHESION (contrainte d'EXCLUSION : X), ce qui remplace l'ancien système de tags.

Enfin, les COUPON sont modélisés comme un référentiel lié aux INSCRIPTION. Une seule base couvre toutes les années de conférence, les périodes et les coordonnées de facturation sont figées au bon endroit. Les requêtes demandées par les trésoriers et organisateurs (montants par année, liste des participants/ateliers, repas de gala avec régimes/allergies, utilisateurs de coupons) se formulent simplement à partir de ces entités et associations.

Modèle logique relationnel (MLD)

PERSONNE(personne_id, prenom, nom, email, web, last_login, pwd_hash, pwd_policy_version)

MEMBRE(numero_membre, #personne_id UNIQUE, date_attribution)

ADHESION(adhesion_id, #personne_id, organisme, debut, fin, tarif)
CHECK : debut < fin / UNIQUE Key(#personne_id, debut)

ADRESSE(adresse_id, #personne_id, type, ligne1, ligne2 NULL, code_postal, ville, pays, is_principale BOOLEAN DEFAULT FALSE, valide_du DATE, valide_au DATE NULL)

CONFERENCE(conf_id, code UNIQUE, libelle, date_debut, date_fin, ville)

EVENEMENT(evt_id, #conf_id, type, libelle, capacite)

ATELIER(atelier_id, #evt_id UNIQUE, theme, creneau, capacite)

COUPON(coupon_code, libelle, type, valeur, cumulable, valide_du, valide_au, nb_max_usages)

INSCRIPTION(inscr_id, #personne_id, #conf_id, statut, type, date_inscr, montant_calcule, #coupon_code NULL, besoins_accessibilite, regime, allergies, autres)

UK : (#personne_id, #conf_id) → pour interdire les doubles inscriptions à la même conférence.

PARTICIPATION_EVT(inscr_id, evt_id, quantite DEFAULT 1)

PRODUIT(produit_id, nom, prix_public DECIMAL(10,2), prix_membre DECIMAL(10,2), categorie, #conf_id NULL)

FACTURE(facture_id, date_facture, code_acces, #personne_id, #adresse_id_source NULL, adresse_nom, adresse_prenom, email, phone_number)

LIGNE_FACTURE(facture_id, ligne_no, #produit_id, qte, prix_unitaire_ttc, reduction_ttc, #inscr_id NULL, #adhesion_id NULL)

PK(facture_id, ligne_no)

CK : X (#inscr_id, #adhesion_id)

PAIEMENT(paiement_id, #facture_id, date_paiement, moyen, ref_transaction, autorisation, virement_ref, montant DECIMAL(10,2))

Partie 2 – Base de données

Q4 - Justification des requêtes

Nous avons choisi de réaliser cinq requêtes qui répondent à des besoins concrets des trésoriers et des organisateurs de la conférence.

R1 – Cette requête permet d’obtenir le nombre de repas pour le gala, avec la spécification des régimes et des allergies, cette requête est orientée pour les organisateurs. Cette requête permet d’éviter les erreurs logistiques.

Par rapport à l’ancien schéma, où chaque conférence contenait des colonnes ajoutées au fil des années, et qui été dupliquée chaque année.

Dans ce modèle, les informations sont centralisées dans une seule table INSCRIPTION, une table PARTICIPATION_EVT et une table EVENEMENT pour le gala relié par les clés étrangères (pas besoin de traiter à la main les colonnes spécifique).

R2 – Cette requête permet de calculer le montant total des cotisation AFIHM versées en fonction des années. Cette requête est orientée pour l’équipe trésorerie. Elle permet de présenter le bilan des cotisations par année, ce qui est utile lors de l’assemblée générale par exemple, ou encore pour suivre les recettes liées aux adhésions (diminution, augmentation).

Initialement, la table PAIEMENT géraient les factures et les paiements. La notion « tag » indiquait si la facture concernait une adhésion ou une inscription à une conférence. Une facture ne pouvait concerner qu’un seul type de produit (adhésion ou inscription). De plus, il n’était pas possible de mixer plusieurs adhésions ou produits sur une seule facture.

Ce modèle permet de distinguer les adhésions et les inscriptions avec une table dédiée. Avec la table LIGNE_FACTURE, plusieurs lignes sur une facture sont autorisées tout en gardant le lien vers l’adhésion facturée.

R3 – Cette requête permet de calculer le montant total des inscriptions en fonction des de la conférence et de l’année. Elle est utile pour l’équipe organisation. Elle permet de suivre les recettes liées aux inscriptions, de comparer les années pour par exemple analyser l’évolution de participation.

Dans le modèle actuel, chaque conférence a sa propre base avec la table PARTICIPATION2024, PARTICIPATION 2025, etc. et la facturation était gérée par la table PAIEMENTS avec des liens tel que « tags » ou « codes ».

Le nouveau modèle permet de référencer toutes les éditions d’une conférence avec une table CONFERENCE dédié et un champs « date_debut ». La table INSCRIPTION relie

chaque personne à une conférence. La table LIGNE_FACTURE permet de retrouver précisément les montants facturés (les réductions s'il y en a) et les factures payées sont identifiées via PAIEMENT.

Grâce à ce modèle, en une seule requête il est possible de calculer les montants par conférence et par année. Alors qu'avec le modèle actuel, il faut d'abord traiter chaque base de conférence séparément, puis recouper les informations avec la table FACTURE via des exports.

R4 – Cette requête permet de donner pour chaque coupon de réduction, le nombre d'utilisateurs et le montant total des inscriptions qui y sont associées. Cela permet d'identifier quel coupon sont réellement utilisés. Utile pour les organisateurs.

Dans les schémas de base, l'information sur les remises était souvent gérée de manière hétérogène (colonnes spécifiques dans la table des participants, tags, champs booléens...). Chaque nouvelle promotion impliquait souvent l'ajout d'une nouvelle colonne ou d'un nouveau code.

Dans le nouveau modèle, les coupons sont centralisés dans une table COUPON, qui relie chaque inscription à un événement avec le champ « coupon_code » et qui stock un montant calculé pour l'inscription avec le champ « montant_calcule ».

R5 – Cette requête permet d'obtenir la liste des participants par atelier, avec leur nom, leur prenom et la quantité (le nombre de place) pour chaque participation afin préparer au mieux les atelier (logistique, ressources, liste d'émergence, etc).

Comme nous l'avion identifié dans les problèmes, la participation aux événements (session, gala, atelier, etc.) était gérée via des colonnes booléennes dans la table des participants (p_nuitdemo, p_gala, p_jcjc, etc.) et chaque conférence avait sa propre version en fonction des années avec l'ajout de colonnes.

Ce nouveau modèle introduit une table EVENELEM, une table ATELIER et une table PARTICIPATION_EVT. Grâce à ça, la requête 5 permet de lister les participants à tous les ateliers de manière flexible. En effet, il n'y a pas besoin d'ajouter une nouvelle colonne pour chaque nouvel atelier, une simple ligne dans la table ATELIER + des lignes dans PARTICIPATION_EVT suffisent. De plus, la requête reste la même d'une année à l'autre.

Pour conclure, ce nouveau modèle répond plus proprement au besoin métier que celui initial.